



Compte Rendu

SAE Exploitation d'une base de données

S2C	KEISER Kenan
S2C	EDOUARD Justin

Partie BD:

1. Identifier les clés étrangères dans le schéma de la base de données.

Les clés étrangères sont :

- codeMembre : dans la table s_adhesion
- codeType : dans la table s_materiel
- idMateriel et numClub : dans la table s_stock
- codeMembre et numClub : dans la table s_location
- numLoc et idMateriel : dans la table s_detaillocation
- numClub : dans la table s_activite
- numActivite et codeMembre : dans la table s_participation

2. [SQL] L'utilisateur « ensinfo » dispose d'une copie des tables de la base. Récupérer ces tables et leurs données depuis ce compte. Ajouter les contraintes de clés primaires et étrangères. Consulter le contenu de chaque table.

```
create table s_club as
select * from ensinfo.S_CLUB;
create table s_membre as
select * from ensinfo.S_MEMBRE;
create table s_adhesion as
select * from ensinfo.S_ADHESION;
create table s_typemateriel as
select * from ensinfo.S_TYPEMATERIEL;
create table s_materiel as
select * from ensinfo.S_MATERIEL;
```

```

create table s_stock as
select * from ensinfo.S_STOCK;
create table s_location as
select * from ensinfo.S_LOCATION;
create table s_detaillocation as
select * from ensinfo.S_DETAILLOCATION;
create table s_activite as
select * from ensinfo.S_ACTIVITE;
create table s_participation as
select * from ensinfo.S_PARTICIPATION;

alter table s_club add primary key(numClub);
alter table s_membre add primary key(codeMembre);
alter table s_typemateriel add primary key(codeType);
alter table s_adhesion add primary key(numAdhesion);
alter table s_adhesion add foreign key(codeMembre) references s_membre(codeMembre);
alter table s_materiel add primary key(idMateriel);
alter table s_materiel add foreign key(codeType) references s_typemateriel(codeType);
alter table s_location add primary key(numLoc);
alter table s_location add foreign key(codeMembre) references s_membre(codeMembre);
alter table s_location add foreign key(numClub) references s_club(numClub);
alter table s_activite add primary key(numActivite);
alter table s_activite add foreign key(numClub) references s_club(numClub);
alter table s_stock add primary key(idMateriel, numClub);
alter table s_stock add foreign key(idMateriel) references s_materiel(idMateriel);
alter table s_stock add foreign key(numClub) references s_club(numClub);
alter table s_detaillocation add primary key(numLoc, idMateriel);
alter table s_detaillocation add foreign key(numLoc) references s_location(numLoc);
alter table s_detaillocation add foreign key(idMateriel) references s_materiel(idMateriel);
alter table s_participation add primary key(numActivite, codeMembre);
alter table s_participation add foreign key(numActivite) references s_activite(numActivite);
alter table s_participation add foreign key(codeMembre) references s_membre(codeMembre);

```

3. [PL/SQL] La colonne REDUCTION de la table S_LOCATION est à NULL. Une mauvaise manipulation a effacé ces données. Ecrire une procédure stockée MAJ_REDUCTION avec un curseur de mise à jour permettant d'indiquer le montant de la réduction (0 ou 0.10) en vérifiant que le membre avait une adhésion en cours de validité au moment de la location.

Create or Replace procedure **reducLocation**

```

is
    cursor estAdherent is select DATELOC, CODEMEMBRE from S_LOCATION;
    locDate DATE;

```

```

    membreC VARCHAR2(10);
    dateDeb DATE;
    dateFin DATE;

begin
    open estAdherent;
    fetch estAdherent into locDate, membreC;
    while estAdherent%FOUND LOOP
        Begin
            SELECT DATEDEBADHESION, DATEFINADHESION into dateDeb, dateFin from S_ADHESION where
            S_ADHESION.CODEMEMBRE = membreC AND locDate between DATEDEBADHESION AND DATEFINADHESION AND
            ROWNUM = 1;

            update S_LOCATION set REDUCTION = 0.1 where CODEMEMBRE = membreC and DATELOC = locDate;
        Exception
            when NO_DATA_FOUND THEN
                UPDATE S_LOCATION
                SET REDUCTION = 0
                WHERE CODEMEMBRE = membreC AND DATELOC = locDate;
        End;
        fetch estAdherent into locDate, membreC;
    END LOOP;
    close estAdherent;
end;

declare
begin
    reducLocation;
end;

```

4. [SQL] Afficher le nom et la quantité du matériel le plus loué en 2023.

```

SELECT nomMateriel, SUM(qte) as quantite FROM s_materiel
INNER JOIN s_detaillocation ON s_materiel.idMateriel = s_detaillocation.idMateriel
INNER JOIN s_location ON s_detaillocation.numLoc = s_location.numLoc
WHERE SUBSTR(TO_DATE(dateLoc), 7, 8) = '23'
GROUP BY nomMateriel
HAVING SUM(qte) = (
    SELECT MAX(quantitef) FROM (
        SELECT SUM(qte) AS quantitef
        FROM s_detaillocation
        INNER JOIN s_location ON s_detaillocation.numLoc = s_location.numLoc
        WHERE SUBSTR(TO_DATE(dateLoc), 7, 8) = '23'
        GROUP BY idMateriel
    )
)

```

Résultat : Sac Randonnée 50L, 16

5. [SQL] Afficher le nombre de locations par type de matériel (libellé).

```
SELECT SUM(numLoc), libelleType
FROM s_detaillocation
INNER JOIN s_materiel ON s_detaillocation.idMateriel = s_materiel.idMateriel
INNER JOIN s_typemateriel ON s_materiel.codeType = s_typemateriel.codeType
GROUP BY libelleType;
```

Résultat :

421, Kayak

331, Vélo

332, Randonnée

380, Escalade

323, Fitness

6. [SQL] Créer une vue S_MontantLocation affichant pour chaque location : son numéro, le montant avant réduction, la réduction, le montant total après réduction. Utiliser cette vue pour afficher les montants de la location numéro 28.

```
CREATE OR REPLACE VIEW S_MontantLocation AS
```

```
SELECT s_location.numLoc, tarifLocation, reduction, (tarifLocation-tarifLocation*reduction) as
tarifLocationApresReduc FROM s_location
```

```
INNER JOIN s_detaillocation ON s_location.numLoc = s_detaillocation.numLoc
```

```
INNER JOIN s_materiel ON s_detaillocation.idMateriel = s_materiel.idMateriel
```

```
GROUP BY s_location.numLoc, tarifLocation, reduction;
```

```
SELECT * FROM S_MontantLocation
```

```
WHERE numLoc = 28;
```

Résultat :

28	250	0.1	225
28	35	0.1	31.5

7. [PL/SQL] Écrire une fonction stockée estEligibleReduction qui prend en paramètre un code membre (codeMembre) et une date (p_date), et retourne 0.10 (10 %) si le membre a une adhésion en cours de validité à cette date, 0 sinon. A l'aide d'une requête SQL, afficher les membres éligibles à une réduction à la date du 1/05/2024.

```
CREATE OR REPLACE FUNCTION estEligibleReduction(p_codeMembre VARCHAR2, p_date DATE) RETURN NUMBER
```

IS

Reduc NUMBER := 0;

Dateannule DATE;

BEGIN

SELECT dateAnnul into Dateannule FROM s_adhesion

WHERE codeMembre = p_codeMembre AND dateDebAdhesion < p_date AND p_date < dateFinAdhesion;

IF Dateannule IS NULL OR p_date < Dateannule THEN

 Reduc := 0.10;

END IF;

RETURN Reduc;

END estEligibleReduction;

SELECT nom, prenom FROM s_membre

WHERE estEligibleReduction(codeMembre, TO_DATE('01/05/24', 'DD/MM/YY')) = 0.10;

Résultat :

Petit, Fabien

Moreau, Hélène

Simon, Julie

Bernard, Marc

Ahmed, Karim

8. [PL/SQL] Écrire une procédure stockée annulerAdhesion qui prend en paramètre un numéro d'adhésion (numAdhesion) et une date d'annulation (dateAnnul), met à jour le statut à 'annulé' et remplit la date d'annulation si l'adhésion est active et que la date est valide (postérieure à dateDebAdhesion et antérieure à dateFinAdhesion). 3

La procédure renvoie dans un paramètre de sortie p_error, le type d'erreur rencontrée :

1 : adhésion inexistante

2 : conditions d'annulation non valides

3 : toute autre erreur.

Utiliser une exception utilisateur pour traiter le cas des conditions d'annulation invalides.

Si pas d'erreur, p_error est à 0.

CREATE OR REPLACE PROCEDURE annulerAdhesion(p_numAdhesion VARCHAR2, p_dateAnnul DATE, p_error out NUMBER)

IS

dateDebut DATE;

dateFin DATE;

Statut VARCHAR2(30);

ErrorAnnulation EXCEPTION;

BEGIN

p_error := 0;

```

SELECT dateDebAdhesion, dateFinAdhesion, statutAdhesion INTO dateDebut, dateFin, Statut FROM
s_adhesion

WHERE numAdhesion = p_numAdhesion;

IF dateDebut < p_dateAnnul AND p_dateAnnul < dateFin AND Statut = 'actif' THEN

    UPDATE s_adhesion SET statutAdhesion = 'annulé', dateAnnul = p_dateAnnul WHERE numAdhesion =
p_numAdhesion;

ELSE

    RAISE ErrorAnnulation;

END IF;

EXCEPTION

    WHEN NO_DATA_FOUND THEN

        p_error := 1;

    WHEN ErrorAnnulation THEN

        p_error := 2;

    WHEN OTHERS THEN

        p_error := 3;

END annulerAdhesion;

DECLARE

    p_error NUMBER;

BEGIN

    annulerAdhesion('ADH015', TO_DATE('01/06/24', 'DD/MM/YY'), p_error);

    DBMS_OUTPUT.PUT_LINE(p_error);

END;

DECLARE

    p_error NUMBER;

BEGIN

    annulerAdhesion('ADH126', TO_DATE('01/06/24', 'DD/MM/YY'), p_error);

    DBMS_OUTPUT.PUT_LINE(p_error);

END;

DECLARE

    p_error NUMBER;

BEGIN

    annulerAdhesion('ADH015', TO_DATE('01/06/27', 'DD/MM/YY'), p_error);

    DBMS_OUTPUT.PUT_LINE(p_error);

END;

```

9. [PL/SQL] Écrire une procédure stockée inscrireActivite qui prend en paramètre un numéro d'activité (numActivite), un code membre (codeMembre), et une date d'inscription (dateInscription), et ajoute une participation si la capacité n'est pas dépassée et que le

membre n'est pas déjà inscrit. Effectuer les vérifications nécessaires sur l'existence de l'activité, du membre concerné, l'inscription déjà existante. Renvoyer dans un paramètre de sortie p_error un code spécifique (ex. p_error vaut 1 si membre inexistant, 2 si activité inexistante, 3 déjà inscrit, 4 capacité dépassée...). Tester tous les cas.

```
create or replace procedure inscrireActivite(numA varchar2, codeM varchar2, dateI date,p_error out number)
is
```

```
    ActiviteC number;
    ActiviteN number;
    testI varchar2(10);
    test2 varchar2(10);
    p_membre EXCEPTION;
    p_activite EXCEPTION;
    p_complet EXCEPTION;
    p_dejaInscrit EXCEPTION;
```

```
begin
    p_error := 0;
    begin
        Select CAPACITE into ActiviteC from S_ACTIVITE WHERE NUMACTIVITE = numA;
        select count(NUMACTIVITE) into ActiviteN from S_PARTICIPATION where NUMACTIVITE = numA;
    exception
        when NO_DATA_FOUND then
            raise p_activite;
    end;
    begin
        select CODEMEMBRE into testI from S_MEMBRE WHERE CODEMEMBRE = codeM;
    exception
        when NO_DATA_FOUND then
            raise p_membre;
    end;
    select NUMACTIVITE into test2 from S_PARTICIPATION WHERE CODEMEMBRE = codeM AND NUMACTIVITE =
numA;
    if testI = codeM and test2 = numA then
        raise p_dejaInscrit;
    end if;
    if ActiviteN >= ActiviteC then
        raise p_complet;
    end if;
```

```
EXCEPTION
```

```
    when p_activite then
        p_error := 2;
```

```

when p_membre then
    p_error := 1;
when p_complet then
    p_error := 4;
when p_dejaInscrit then
    p_error := 3;
when NO_DATA_FOUND then
    INSERT INTO S_PARTICIPATION VALUES(numA,codeM,datel);
end;

declare
p_error number;
p_error1 number;
p_error2 number;
begin
inscrireActivite('ACT001','MEM100',TO_DATE('24/02/23','dd/mm/yy'),p_error);
DBMS_OUTPUT.PUT_LINE(p_error);
inscrireActivite('ACT100','MEM001',TO_DATE('24/02/23','dd/mm/yy'),p_error1);
DBMS_OUTPUT.PUT_LINE(p_error1);
inscrireActivite('ACT001','MEM001',TO_DATE('24/02/23','dd/mm/yy'),p_error2);
DBMS_OUTPUT.PUT_LINE(p_error2);
end;

```

10. [SQL] La table S_LOCATION est complétée par une liste de location. Réaliser les insertions à partir du fichier insertComplementLocation.

```

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (51, 'MEM012',
TO_DATE('01/05/2023', 'DD/MM/YYYY'), 0, 'terminée', 'SA2');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (52, 'MEM013',
TO_DATE('01/06/2023', 'DD/MM/YYYY'), 0, 'terminée', 'SA3');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (53, 'MEM014',
TO_DATE('01/07/2023', 'DD/MM/YYYY'), 0, 'terminée', 'SA4');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (54, 'MEM015',
TO_DATE('01/08/2023', 'DD/MM/YYYY'), 0, 'terminée', 'SA5');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (55,
'MEM009', TO_DATE('01/09/2023', 'DD/MM/YYYY'), 0, 'terminée', 'SA6');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (56,
'MEM010', TO_DATE('01/10/2023', 'DD/MM/YYYY'), 0, 'terminée', 'SA7');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (57, 'MEM011',
TO_DATE('01/11/2023', 'DD/MM/YYYY'), 0, 'terminée', 'SA8');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (58, 'MEM012',
TO_DATE('01/12/2023', 'DD/MM/YYYY'), 0, 'terminée', 'SA5');

```


INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (59, 'MEMOI3', TO_DATE('01/01/2024', 'DD/MM/YYYY'), 0, 'terminée', 'SA9');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (60, 'MEMOI4', TO_DATE('20/07/2023', 'DD/MM/YYYY'), 0, 'terminée', 'SAIO');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (61, 'MEMOI5', TO_DATE('25/08/2023', 'DD/MM/YYYY'), 0, 'terminée', 'SAI');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (62, 'MEMOO1', TO_DATE('30/09/2023', 'DD/MM/YYYY'), 0, 'terminée', 'SA2');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (63, 'MEMOO2', TO_DATE('05/10/2023', 'DD/MM/YYYY'), 0, 'terminée', 'SA3');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (64, 'MEMOO3', TO_DATE('10/11/2023', 'DD/MM/YYYY'), 0, 'terminée', 'SA4');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (65, 'MEMOO4', TO_DATE('15/12/2023', 'DD/MM/YYYY'), 0, 'terminée', 'SAI');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (66, 'MEMOO5', TO_DATE('20/01/2024', 'DD/MM/YYYY'), 0, 'terminée', 'SA2');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (67, 'MEMOO6', TO_DATE('25/02/2024', 'DD/MM/YYYY'), 0, 'terminée', 'SA3');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (68, 'MEMOO7', TO_DATE('30/03/2024', 'DD/MM/YYYY'), 0, 'terminée', 'SA4');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (69, 'MEMOO8', TO_DATE('05/04/2024', 'DD/MM/YYYY'), 0, 'terminée', 'SA5');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (70, 'MEMOO9', TO_DATE('10/05/2024', 'DD/MM/YYYY'), 0, 'terminée', 'SA6');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (71, 'MEMOIO', TO_DATE('15/06/2024', 'DD/MM/YYYY'), 0, 'terminée', 'SA7');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (72, 'MEMOII', TO_DATE('01/09/2023', 'DD/MM/YYYY'), 0, 'terminée', 'SA8');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (73, 'MEMOI2', TO_DATE('01/10/2023', 'DD/MM/YYYY'), 0, 'terminée', 'SA9');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (74, 'MEMOO6', TO_DATE('01/11/2023', 'DD/MM/YYYY'), 0, 'terminée', 'SA7');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (75, 'MEMOO7', TO_DATE('01/12/2023', 'DD/MM/YYYY'), 0, 'terminée', 'SA8');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (76, 'MEMOO8', TO_DATE('01/01/2024', 'DD/MM/YYYY'), 0, 'terminée', 'SA9');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (77, 'MEMOO9', TO_DATE('01/02/2024', 'DD/MM/YYYY'), 0, 'terminée', 'SAIO');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (78, 'MEMOIO', TO_DATE('01/03/2024', 'DD/MM/YYYY'), 0, 'terminée', 'SAI');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (79, 'MEMOII', TO_DATE('01/04/2024', 'DD/MM/YYYY'), 0, 'terminée', 'SA3');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (80, 'MEMOI2', TO_DATE('01/05/2024', 'DD/MM/YYYY'), 0, 'terminée', 'SA2');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (81, 'MEMOI3', TO_DATE('01/06/2024', 'DD/MM/YYYY'), 0, 'terminée', 'SA3');

```

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (82, 'MEMO14',
TO_DATE('01/07/2024', 'DD/MM/YYYY'), 0, 'terminée', 'SA4');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (83, 'MEMO15',
TO_DATE('01/08/2024', 'DD/MM/YYYY'), 0, 'terminée', 'SA5');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (84, 'MEMO13',
TO_DATE('01/09/2024', 'DD/MM/YYYY'), 0, 'terminée', 'SA6');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (85,
'MEMO06', TO_DATE('01/12/2023', 'DD/MM/YYYY'), 0, 'terminée', 'SA7');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (86,
'MEMO07', TO_DATE('01/01/2024', 'DD/MM/YYYY'), 0, 'terminée', 'SA8');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (87,
'MEMO08', TO_DATE('20/07/2023', 'DD/MM/YYYY'), 0, 'terminée', 'SA5');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (88,
'MEMO09', TO_DATE('25/08/2023', 'DD/MM/YYYY'), 0, 'terminée', 'SA9');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (89,
'MEMO10', TO_DATE('30/09/2023', 'DD/MM/YYYY'), 0, 'terminée', 'SA10');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (90, 'MEMO11',
TO_DATE('05/10/2023', 'DD/MM/YYYY'), 0, 'terminée', 'SA10');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (91, 'MEMO12',
TO_DATE('10/11/2023', 'DD/MM/YYYY'), 0, 'terminée', 'SA8');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (92, 'MEMO13',
TO_DATE('15/12/2023', 'DD/MM/YYYY'), 0, 'terminée', 'SA1');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (93, 'MEMO14',
TO_DATE('20/01/2024', 'DD/MM/YYYY'), 0, 'terminée', 'SA2');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (94, 'MEMO15',
TO_DATE('25/02/2024', 'DD/MM/YYYY'), 0, 'terminée', 'SA3');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (95,
'MEMO06', TO_DATE('30/03/2024', 'DD/MM/YYYY'), 0, 'terminée', 'SA4');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (96,
'MEMO07', TO_DATE('05/04/2024', 'DD/MM/YYYY'), 0, 'terminée', 'SA5');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (97,
'MEMO08', TO_DATE('10/05/2024', 'DD/MM/YYYY'), 0, 'terminée', 'SA2');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (98,
'MEMO09', TO_DATE('15/06/2024', 'DD/MM/YYYY'), 0, 'terminée', 'SA3');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (99,
'MEMO10', TO_DATE('01/09/2023', 'DD/MM/YYYY'), 0, 'terminée', 'SA4');

INSERT INTO S_LOCATION (numLoc, codeMembre, dateLoc, reduction, statut, numClub) VALUES (100,
'MEMO11', TO_DATE('01/10/2023', 'DD/MM/YYYY'), 0, 'terminée', 'SA5');

```

II. [SQL] Afficher le nombre total de locations par club en 2023. Répéter l'opération pour 2024.

```

SELECT SUM(numLoc), nomClub FROM s_location
INNER JOIN s_club ON s_location.numClub = s_club.numClub
WHERE SUBSTR(TO_DATE(dateLoc), 7, 8) = '23'
GROUP BY nomClub;

```

Résultat :

266, SportAventure 1
183, SportAventure 3
361, SportAventure 5
369, SportAventure 8
165, SportAventure 2
274, SportAventure 4
120, SportAventure 6
203, SportAventure 9
264, SportAventure 7
288, SportAventure 10

```
SELECT SUM(numLoc), nomClub FROM s_location  
INNER JOIN s_club ON s_location.numClub = s_club.numClub  
WHERE SUBSTR(TO_DATE(dateLoc), 7, 8) = '24'  
GROUP BY nomClub;
```

Résultat :

148, SportAventure 1
347, SportAventure 5
193, SportAventure 8
408, SportAventure 2
321, SportAventure 4
183, SportAventure 6
200, SportAventure 9
144, SportAventure 10
120, SportAventure 7
493, SportAventure 3

12. [SQL] Afficher le nombre de membres par club trié par ville (ville, nomclub et nombre)

```
SELECT nomClub, ville, COUNT(DISTINCT(s_adhesion.codeMembre)) as nombre FROM s_club  
INNER JOIN s_location ON s_club.numClub = s_location.numClub  
INNER JOIN s_adhesion ON s_location.codeMembre = s_adhesion.codeMembre  
GROUP BY nomClub, ville  
ORDER BY ville;
```

Résultat :

SportAventure 9, Biarritz, 7
SportAventure 5, Bordeaux, 10
SportAventure 4, Grenoble, 9
SportAventure 6, Lille, 6
SportAventure 3, Marseille, 8
SportAventure 8, Nice, 8

SportAventure 2, Lyon, 8

SportAventure 1, Paris, 8

SportAventure 10, Strasbourg, 7

SportAventure 7, Toulouse, 6

13. [PL/SQL] Écrire un bloc PL/SQL affichant les détails d'une location (matériel, prix de location, quantité) à partir de son numéro saisi au clavier. Gérer le cas où la location n'existe pas. Afficher le total avant et après réduction.

```
declare
num number;
code varchar2(10);
dateL date;
status varchar2(20);
club varchar2(20);
reduc number;
quantite number;
chose varchar2(20);
nom varchar2(50);
prix number;
total number;
cursor materiel is select IDMATERIEL,QTE FROM S_DETAILLOCATION where NUMLOC = num;
begin
total := 0;
num := &location;

select CODEMEMBRE, DATELOC, STATUT, NUMCLUB, REDUCTION into code, dateL, status, club, reduc from
S_LOCATION WHERE NUMLOC = num;

DBMS_OUTPUT.PUT_LINE('Détails de la location numéro ' || num);
DBMS_OUTPUT.PUT_LINE('-----');
DBMS_OUTPUT.PUT_LINE('Code Membre : ' || code);
DBMS_OUTPUT.PUT_LINE('Date de location : ' || dateL);
DBMS_OUTPUT.PUT_LINE('Statut : ' || status);
DBMS_OUTPUT.PUT_LINE('Club : ' || club);
DBMS_OUTPUT.PUT_LINE('Reduction : ' || (reduc*10)|| '%');
DBMS_OUTPUT.PUT_LINE('-----');
DBMS_OUTPUT.PUT_LINE('Matériel loué :');
DBMS_OUTPUT.PUT_LINE('-----');
DBMS_OUTPUT.PUT_LINE('Nom du matériel | Prix unitaire | Quantité | Sous-total');
DBMS_OUTPUT.PUT_LINE('-----');

open materiel;
fetch materiel into chose, quantite;
while materiel%found loop
```

```

select TARIFLOCATION, NOMMATERIEL into prix, nom from S_MATERIEL WHERE IDMATERIEL = chose;
DBMS_OUTPUT.PUT_LINE(nom || ' ' || prix || ' ' || quantite || ' ' || (prix*quantite));
total := total + (prix*quantite);
fetch materiel into chose, quantite;
end loop;
close materiel;
DBMS_OUTPUT.PUT_LINE('-----');
DBMS_OUTPUT.PUT_LINE('Total avant réduction : ' || total);
DBMS_OUTPUT.PUT_LINE('Montant de la réduction : ' || reduc);
DBMS_OUTPUT.PUT_LINE('Total après réduction : ' || (total - (total * reduc)));

exception
when NO_DATA_FOUND then
DBMS_OUTPUT.PUT_LINE('Pas de location au numero ' || num);
end;

```

14. [PL/SQL] Écrire un bloc PL/SQL affichant le catalogue de matériel par type avec :

- Type de matériel
- Nom du matériel, prix neuf, année de fabrication, total des locations.

DECLARE

CURSOR TypeL IS

SELECT libelleType FROM s_typemateriel;

CURSOR TypeL_info(p_libel VARCHAR2) IS

SELECT nomMateriel, tarifLocation, anneeFabrication, COUNT(qte) as total FROM s_typemateriel
INNER JOIN s_materiel ON s_typemateriel.codeType = s_materiel.codeType
INNER JOIN s_detaillocation ON s_materiel.idMateriel = s_detaillocation.idMateriel
WHERE libelleType = p_libel
GROUP BY nomMateriel, tarifLocation, anneeFabrication;

BEGIN

dbms_output.put_line('CATALOGUE DE MATERIEL PAR TYPE');

dbms_output.put_line('=====');

FOR TypeLloop IN TypeL LOOP

dbms_output.put_line('');

dbms_output.put_line('TYPE: ' || TypeLloop.libelleType);

dbms_output.put_line('-----');

dbms_output.put_line('Nom du matériel | Prix | Année | Nombre de locations');

dbms_output.put_line('----- | ----- | ----- | -----');

FOR TypeL_infoloop IN TypeL_info(TypeLloop.libelleType) LOOP

dbms_output.put_line(TypeL_infoloop.nomMateriel || ' ' || TypeL_infoloop.tarifLocation || ' ' ||
TypeL_infoloop.anneeFabrication || ' ' || TypeL_infoloop.total);

```
END LOOP;  
END LOOP;  
END;
```

15. [PL/SQL] Écrire un bloc PL/SQL qui pour un numéro de club et une année donnés affiche pour ce club :

- le nombre total d'activités organisées dans l'année
- la capacité totale
- le taux de remplissage moyen des activités (pourcentage de participants par rapport à la capacité).
- le nombre de participants
- le nombre de membres distincts ayant participé à au moins une activité.

DECLARE

```
nClub VARCHAR2(3);  
Ann NUMBER;  
numactT NUMBER;  
capaT NUMBER;  
tauxM NUMBER;  
nbPartici NUMBER;  
nbDist NUMBER;
```

BEGIN

```
nClub := &numClub;  
Ann := &Annee;
```

```
SELECT SUM(capacite) INTO capaT FROM s_activite  
WHERE numClub = nClub AND SUBSTR(dateActivite, 7, 8) = SUBSTR(Ann, 3, 4);
```

```
SELECT COUNT(DISTINCT(s_participation.numActivite)), (COUNT(codeMembre)/capaT),  
COUNT(codeMembre), COUNT(DISTINCT(codeMembre)) INTO numactT, tauxM, nbPartici, nbDist FROM s_club  
INNER JOIN s_activite ON s_club.numClub = s_activite.numClub  
INNER JOIN s_participation ON s_activite.numActivite = s_participation.numActivite  
WHERE s_club.numClub = nClub AND SUBSTR(dateActivite, 7, 8) = SUBSTR(Ann, 3, 4);  
  
dbms_output.put_line('Statistiques pour le club ' || nClub || ' en ' || Ann);  
dbms_output.put_line('-----');  
dbms_output.put_line('Nombre total d'activités organisées : ' || numactT);  
dbms_output.put_line('Capacité totale : ' || capaT);  
dbms_output.put_line('Taux de remplissage moyen : ' || tauxM*100 || '%');  
dbms_output.put_line('Nombre total de participants : ' || nbPartici);  
dbms_output.put_line('Nombre de membres distincts participants : ' || nbDist);  
END;
```

